



Valid Parentheses

Company: Jp morgan

Year: 2023

Difficulty: Easy

Topic: Stack, Strings

Source: [LeetCode 20 – Valid Parentheses](#)

Problem Description

You are given a string `s` containing only the following bracket characters:

`(){}[]`

You need to determine whether the brackets in the string are **valid**.

A string is valid if:

1. Every opening bracket has a matching closing bracket of the **same type**.
2. Brackets are closed in the **correct order**.
3. Every closing bracket has a corresponding opening bracket.

For example:

`()`

is valid because (is correctly closed by).

Similarly:

`([])`

is valid because every bracket is correctly matched and nested.

Input Format

- The first line contains a string `s` consisting only of the characters (,), { , }, [, and] .

Output Format

- Print true if the string contains valid and correctly ordered parentheses.
- Print false if the brackets are not valid.

Constraints

- $1 \leq s.length \leq 10^4$
- s consists only of the characters ()[]{}.

Example 1

Input:

s = "()"

Output:

true

Explanation

The opening bracket (is correctly closed by).

Example 2

Input:

s = "()[]{}"

Output:

true

Explanation

All three pairs are correctly matched:

()

[]

{}

Therefore, the string is valid.

Example 3

Input:

s = "("

Output:

false

Explanation

(must be closed by), but the string contains].

Therefore, the string is invalid.

Example 4

Input:

s = "([])"

Output:

true

Explanation

The brackets are correctly nested:

```
(  
  [  
  ]  
)
```

Therefore, the string is valid.

Example 5

Input:

s = "([)]"

Output:

false

Explanation

The brackets are not closed in the correct order.

The [should be closed before (, but) appears first.

Therefore, the string is invalid.

Approach to Solve This Problem

The best approach is to use a **Stack**.

The important idea is that brackets follow a **Last In, First Out (LIFO)** order.

For example:

(({}))

When we see opening brackets, we put them into the stack:

(

([

([{

When we encounter a closing bracket, it must match the **most recently opened bracket**.

For example:

{

}

The { is at the top of the stack, so } correctly matches it.

Bracket Mapping

We can use a mapping such as:

(→)

[→]

{ → }

When an opening bracket is found, push it onto the stack.

When a closing bracket is found:

1. Check whether the stack is empty.
2. Check whether the top opening bracket matches the current closing bracket.
3. If it does not match, return false.
4. If it matches, remove the opening bracket from the stack.

At the end, the stack must be empty.

Dry Run

Consider:

s = "([)]"

Step 1

Character:

(

Opening bracket → push into stack.

Stack: (

Step 2

Character:

[



Opening bracket → push.

Stack: ([

Step 3

Character:

]

Top of stack is [.

[matches], so remove it.

Stack: (

Step 4

Character:

)

Top of stack is (.

(matches), so remove it.

Stack: empty

The stack is empty at the end, so:

true

Java Solution

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
String s = sc.nextLine();
```

```
Stack<Character> stack = new Stack<>();
```

```
for (char ch : s.toCharArray()) {
```

```
    // Opening bracket
```

```
    if (ch == '(' || ch == '[' || ch == '{') {
```

```
        stack.push(ch);
```

```
    }
```

```
    // Closing bracket
```

```
    else {
```

```
        if (stack.isEmpty()) {
```

```
            System.out.println(false);
```

```
            return;
```

```
        }
```

```
        char top = stack.pop();
```

```
        if ((ch == ')' && top != '(') ||
```

```
            (ch == ']' && top != '[') ||
```

```
            (ch == '}' && top != '{')) {
```

```
        System.out.println(false);

        return;
    }
}

System.out.println(stack.isEmpty());
}
}
```

Solution:

1. **Read the input:** main() reads the string s, matching the Input Format.
2. **Create a stack:** Create a Stack<Character> to store opening brackets. The stack follows the Last In, First Out (LIFO) principle.
3. **Traverse the string:** Process every character of s from left to right.
4. **Handle opening brackets:** If the current character is (, [, or {, push it onto the stack because it needs to be matched with a future closing bracket.
5. **Handle closing brackets:** If the current character is),], or }, first check whether the stack is empty. If it is empty, there is no corresponding opening bracket, so print false and stop.
6. **Check the matching bracket:** Remove the top element from the stack and check whether it matches the current closing bracket. For example,) must match (,] must match [, and } must match {. If the brackets do not match, print false and stop.
7. **Check the remaining stack:** After processing all characters, the stack must be empty. If it is empty, every opening bracket was correctly closed, so print true. Otherwise, some opening brackets were not closed, so print false.

Complexity Analysis



- **Time Complexity:** $O(N)$ — each character is processed once.
- **Space Complexity:** $O(N)$ — in the worst case, all characters can be opening brackets and stored in the stack.